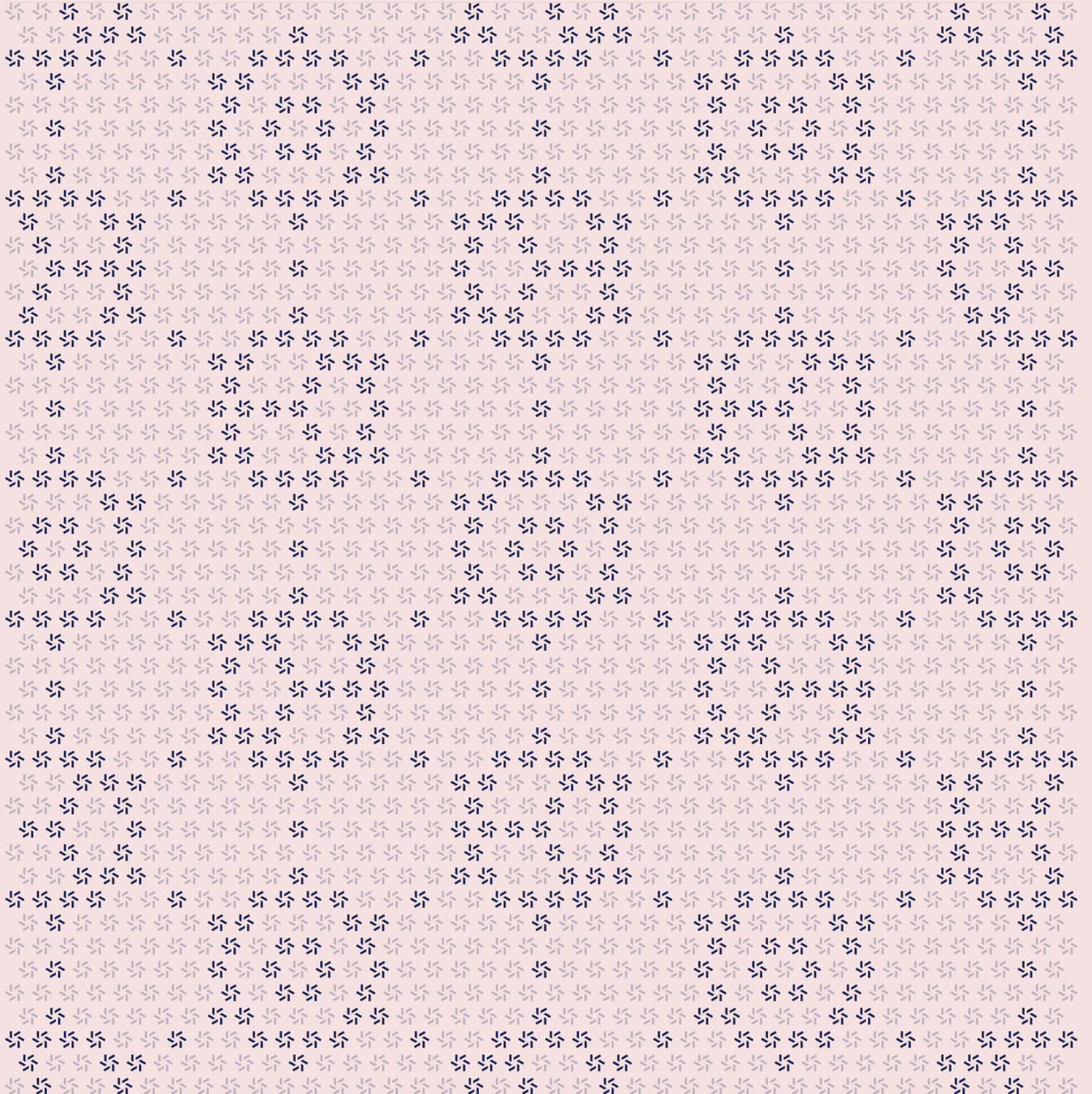


July 31, 2026

Flare Solana OFT

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6
2. Introduction	7
2.1. About Flare Solana OFT	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. Oversized compose payloads can make Solana delivery impossible after source tokens are debited	11
3.2. A zero Solana peer can burn tokens without delivering them	13
3.3. A zero recipient can strand tokens in an inaccessible Solana account	15
3.4. Single-step admin transfers can assign OFT control to an unusable address	17
3.5. <code>quote_send</code> encodes an LD amount as SD	19
3.6. Mismatched cross-chain rate limits can leave transfers pending until reconfiguration	21

3.7.	Maximum sendable amount returned by quote_of_t does not match the actual limit	23
3.8.	Insufficient test coverage	25
4.	Threat model	27
4.1.	EVM	27
4.2.	Solana	48
5.	Assessment Results	71
5.1.	Disclaimer	71

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Flare Ecosystems Ltd. from July 22nd to July 27th, 2026. During this engagement, Zellic reviewed Flare Solana OFT's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Do the EVM and Solana OFT implementations preserve FXRP accounting across lock/unlock and burn/mint transfers?
 - Could an unauthorized user or forged cross-chain message mint, unlock, redirect, or withdraw FXRP?
 - Can users bypass pausing, fees, slippage protection, or per-route rate limits on either side of the OFT mesh?
 - Could malformed messages, incompatible destination constraints, or inconsistent route configuration leave transferred funds stranded?
 - Are administrative, LayerZero delegate, pauser, peer-configuration, fee-withdrawal, and upgrade privileges appropriately restricted?
 - Do quoting and sending produce consistent amounts, fees, messages, and transfer limits across supported decimal configurations?
 - Does the Hyperliquid composer safely forward received assets and handle failed HyperCore transfers?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

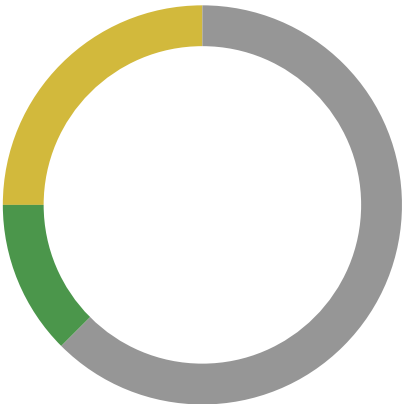
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Flare Solana OFT files, we discovered eight findings. No critical issues were found. Two findings were of medium impact, one was of low impact, and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	2
Low	1
Informational	5



2. Introduction

2.1. About Flare Solana OFT

Flare Ecosystems Ltd. contributed the following description of Flare Solana OFT:

The FAssets OFT layer bridges FAssets (FXRP) across chains using LayerZero's official OFT (Omnichain Fungible Token) standard. FXRP is the FAsset representation of XRP, a collateral-backed token minted on Flare. The OFT layer lets FXRP move between Flare and other chains: an adapter on the FXRP home chain locks and unlocks the canonical token, and native mint/burn OFT contracts represent FXRP on the destination chains. This is a standard OFT deployment, not a custom bridge. This engagement adds a native Solana OFT program - built on LayerZero's official Solana OFT reference implementation - so that FXRP can move to and from Solana, and adds pausing and per-route rate limiting on both the EVM and Solana sides. The pausing and rate limiting are the only Flare-specific additions on top of the standard OFT.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We

also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Flare Solana OFT Files

Types	Rust, Solidity
Platforms	EVM-compatible, Solana
Target	fassets
Repository	https://gitlab.com/flarenetwork/fassets/fasset-oftadapter ↗
Version	399483f1d20b860a64a16deb9cbb31f5481ba580
Programs	programs/oft/**/*. contracts/*.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.4 person-weeks. The assessment was conducted by two consultants over the course of one calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
↗ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filippo Cremonese
↗ Engineer
fcremo@zellic.io ↗

Ding Jie
↗ Engineer
jie@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 22, 2026 Start of primary review period

July 27, 2026 End of primary review period

3. Detailed Findings

3.1. Oversized compose payloads can make Solana delivery impossible after source tokens are debited

Target	FAsset OFT mesh		
Category	Integration Risks	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The EVM-side FAssetOFT and FAssetOFTAdapter contracts inherit LayerZero's standard OFT message construction without imposing a Solana-specific limit on `SendParam.composeMsg`. Consequently, a source transaction can debit tokens and send a LayerZero message that is too large to include in a Solana transaction.

When a compose payload is present, the Solana OFT message encoder appends it after a 72-byte prefix:

```
let mut encoded = Vec::with_capacity(72 + msg.len()); // 32 + 8 + 32
encoded.extend_from_slice(&send_to);
encoded.extend_from_slice(&amount_sd.to_be_bytes());
encoded.extend_from_slice(sender.to_bytes().as_ref());
encoded.extend_from_slice(&msg);
```

Both `lz_receive_types` and `lz_receive` receive the complete OFT message as part of an Anchor-serialized `LzReceiveParams`. With an empty `extra_data` field and an `N`-byte compose payload, the resulting instruction data is `164 + N` bytes:

```
8-byte Anchor discriminator
+ 4-byte source EID
+ 32-byte sender
+ 8-byte nonce
+ 32-byte GUID
+ 4-byte message length
+ (72 + N)-byte OFT message
+ 4-byte extra-data length
```

For example, a 1,024-byte compose payload produces a 1,096-byte OFT message and 1,188 bytes of `lz_receive` instruction data. Solana limits an entire serialized transaction—including signatures, account keys, the recent blockhash, and instruction metadata—to 1,232 bytes. The instruction data therefore leaves insufficient space for even one signature, making the transaction impossible to construct.

Impact

The source transaction can complete successfully before this incompatibility is encountered. FAssetOFTAdapter locks the debited tokens, while FAssetOFT burns them, and the LayerZero packet is emitted. On Solana, an executor cannot construct either the account-resolution or receive transaction, regardless of the supplied execution budget.

The message cannot be delivered, and LayerZero does not automatically reverse the source-chain debit. The transferred tokens therefore remain stranded until the OFT program, executor, or protocol is upgraded with a compatible delivery or recovery mechanism. Although a user generally controls their own compose payload, integrations might inadvertently produce an undeliverable transfer without receiving an actionable source-side error.

Recommendations

Enforce a conservative maximum encoded-message or compose-payload length for every Solana destination EID before debiting tokens on every source chain. Apply the same validation during quoting so that quotes and sends accept identical payloads.

Derive the maximum from a fully serialized executor transaction containing the actual receive accounts and instructions. Reserve space for all variable fields and include a safety margin. If a reliable bound cannot be guaranteed, consider disabling compose messages on routes to Solana. If the risk is accepted, it should be clearly documented to reduce the probability of users producing undeliverable payloads.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd., and a fix was implemented in commit [6aaa77fc](#).

3.2. A zero Solana peer can burn tokens without delivering them

Target	OFT Program		
Category	Coding Mistakes	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The SetPeerConfig instruction accepts any 32-byte peer address, including an all-zero value:

```
match params.config.clone() {
  PeerConfigParam::PeerAddress(peer_address) => {
    ctx.accounts.peer.peer_address = peer_address;
  },
  // ...
}
```

Moreover, the instruction uses `init_if_needed`. Configuring fees, enforced options, or rate limits before setting `PeerAddress` therefore creates a `PeerConfig` whose peer address remains `[0u8; 32]`.

The send instruction validates the peer PDA but does not require its address to be nonzero. It passes the configured value directly to the LayerZero Endpoint:

```
EndpointSendParams {
  dst_eid: params.dst_eid,
  receiver: ctx.accounts.peer.peer_address,
  message: msg_codec::encode(
    params.to,
    amount_sd,
    ctx.accounts.signer.key(),
    &params.compose_msg,
  ),
  // ...
}
```

The pinned LayerZero Endpoint allows a zero-address remote: `init_nonce` accepts any `remote_oapp`, and neither the Endpoint nor ULN send logic rejects a zero receiver. Once the delegate initializes the nonce accounts for this route, the source send succeeds and emits a packet addressed to `[0u8; 32]`, which cannot invoke a valid remote OFT. If those nonce accounts are absent, the send fails and the token debit is rolled back.

Impact

An incomplete or erroneous administrative configuration can expose an apparently usable route whose sends lock tokens in the adapter escrow or burn them in the native OFT without delivering them to a destination OFT. LayerZero does not automatically reverse a successful source-chain debit, and correcting the peer later does not change the receiver encoded in an existing packet.

The issue only manifests in case of an administrative misconfiguration. However, once the route is configured, any user could submit a transfer that would be left stranded.

Recommendations

Require `peer.peer_address != [0u8; 32]` in `send` before debiting tokens. Apply the same check during quoting so that quotes and sends accept the same routes.

If a zero peer is not required to disable a route, also reject it in `set_peer_config`. Otherwise, allow it as an administrative disabled state while ensuring that no user-facing operation can use the route.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd., and a fix was implemented in commit [5ef2d492](#).

configuration does not alter the token account's authority.

Users generally control the recipient encoded in their own messages. However, a frontend or integration error could cause users to irreversibly strand transferred tokens without receiving an actionable source-side error.

Recommendations

Reject an all-zero recipient before debiting tokens on every source chain. Apply the same validation during quoting so that quotes and sends accept the same recipient values.

As defense in depth, also reject a zero recipient in the Solana receive instruction. This destination-side check does not replace source-side validation, because rejecting an existing message after its source debit would leave the transfer pending until a recovery mechanism is available.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd., and fixes were implemented in the following commits:

- [a3b1e929](#) ↗
- [0687be79](#) ↗

3.4. Single-step admin transfers can assign OFT control to an unusable address

Target	OFT Program		
Category	Code Maturity	Severity	Low
Likelihood	Low	Impact	Informational

Description

The SetOFTConfig instruction transfers the OFT administrator role by immediately overwriting `oft_store.admin` with an arbitrary public key:

```
match params.clone() {
  SetOFTConfigParams::Admin(admin) => {
    ctx.accounts.oft_store.admin = admin;
  },
  // ...
}
```

The current administrator is required to authorize this transaction, but the proposed administrator does not need to sign or otherwise demonstrate control of the supplied key. The program has no pending-administrator state, acceptance step, or validation against `[0u8; 32]`, which corresponds to Solana's System Program address.

Consequently, a typographical error, incorrect address, or zero address takes effect immediately. After the transaction completes, the previous administrator no longer satisfies the `has_one = admin` constraint and cannot correct the assignment.

Impact

Assigning the role to an unusable address can permanently disable ordinary administrative operations, including changing the administrator and LayerZero delegate, configuring fees and pause roles, updating peer configuration, and withdrawing accumulated fees. Recovery would require an upgrade of the OFT program.

The issue requires an error by the current administrator and cannot be triggered by an unprivileged user. It therefore represents an operational safeguard deficiency rather than an attacker-controlled privilege escalation.

Recommendations

Consider implementing a two-step administrator transfer. The current administrator should nominate a `pending_admin`, and that address should explicitly accept the role in a separate transaction before `oft_store.admin` is updated. Allow the current administrator to cancel a pending transfer.

Reject `[0u8; 32]` as both the proposed and accepted administrator. Emit events for nomination, cancellation, and acceptance so that administrative transitions can be monitored.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd..

3.5. quote_send encodes an LD amount as SD

Target	OFT Program		
Category	Coding Mistakes	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The OFT message format stores transfer amounts in shared decimals (SD). However, the QuoteSend instruction passes amount_received_ld, which is denominated in local decimals (LD), directly to msg_codec::encode:

```
message: msg_codec::encode(
  params.to,
  amount_received_ld,
  Pubkey::default(),
  &params.compose_msg,
),
```

The actual Send instruction first converts the same value to shared decimals:

```
let amount_sd = ctx.accounts.oft_store.ld2sd(amount_received_ld);

// ...

message: msg_codec::encode(
  params.to,
  amount_sd,
  ctx.accounts.signer.key(),
  &params.compose_msg,
),
```

Consequently, when local and shared decimals differ, quote_send submits a different amount value to the LayerZero quoting path than send subsequently includes in the transmitted message.

Impact

The documented FXRP configuration uses six local decimals and six shared decimals, resulting in an ld2sd_rate of one. The quoted and transmitted amount values are therefore identical for the intended deployment. The amount occupies eight bytes in either representation, so this

discrepancy does not change the message length.

Therefore, this issue does not manifest as unintended behavior, but the implementation is internally inconsistent. A deployment with different local and shared decimals would quote a message whose contents do not faithfully represent the message later sent.

Recommendations

Convert `amount_received_ld` with `oft_store.ld2sd` in `quote_send` and pass the resulting `amount_sd` to `msg_codec::encode`. This keeps message construction consistent between quoting and sending for every supported decimal configuration.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd., and a fix was implemented in commit [46ac0711](#).

3.6. Mismatched cross-chain rate limits can leave transfers pending until reconfiguration

Target	FAsset OFT mesh		
Category	Integration Risks	Severity	Informational
Likelihood	Low	Impact	Informational

Description

Outgoing and incoming rate limiters are configured and enforced independently on each side of a cross-chain route. The source Send instruction consumes the destination peer's outgoing capacity:

```
if let Some(rate_limiter) = ctx.accounts.peer.outbound_rate_limiter.as_mut() {
    rate_limiter.try_consume(amount_received_id)?;
}
```

The destination LzReceive instruction separately consumes the source peer's incoming capacity:

```
if let Some(rate_limiter) = ctx.accounts.peer.inbound_rate_limiter.as_mut() {
    rate_limiter.try_consume(amount_received_id)?;
}
```

The configuration process does not require the source's maximum outgoing capacity to be less than or equal to the destination's maximum incoming capacity. For example, a route can be configured as follows:

```
Source outgoing capacity    = 11
Destination incoming capacity = 10
Transfer amount             = 11
```

The source accepts the transfer, debits the tokens, and emits the LayerZero packet because the amount does not exceed its outgoing capacity. The destination rejects the message because the amount exceeds its incoming capacity. Since the destination bucket can never contain more than ten units, waiting for it to refill cannot make this message executable.

Impact

Tokens are locked in the source adapter or burned by the source native OFT while the LayerZero message remains pending on the destination. Delivery requires an administrator to increase or disable the destination incoming limit and retry the message.

The issue results from inconsistent administrative configuration rather than an unprivileged action. Independent rate limiters can also have temporarily different available capacity during normal operation, so the source cannot reliably enforce the destination's current state on-chain.

Recommendations

Validate rate-limit configurations in deployment and wiring tooling. For each route, ensure that the source's maximum outgoing capacity does not exceed the destination's maximum incoming capacity, and retain a suitable operational margin.

Monitor destination executions for rate-limit failures and document a procedure for temporarily increasing or disabling the incoming limiter before retrying affected messages.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd..

3.7. Maximum sendable amount returned by quote_of_t does not match the actual limit

Target	OFT Program		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

The QuoteOFT instruction returns `u64::MAX` as the maximum transferable amount regardless of the destination peer's outbound rate limiter:

```
let oft_limits = OFTLimits {
    min_amount_ld: 0,
    max_amount_ld: 0xffffffffffffffff,
};
```

The actual Send instruction consumes the post-fee transfer amount from that limiter and rejects a transfer when insufficient capacity remains:

```
if let Some(rate_limiter) = ctx.accounts.peer.outbound_rate_limiter.as_mut() {
    rate_limiter.try_consume(amount_received_ld)?;
}
```

Consequently, `quote_of_t` can report that an amount is within the OFT limits even though an immediate send of that amount would fail with `RateLimitExceeded`.

Impact

Applications that use `oft_limits.max_amount_ld` to validate transfer amounts or construct user interfaces can present unavailable capacity to users. A subsequent send fails and incurs an avoidable failed transaction.

The rate limit itself cannot be bypassed, and the failed send reverts the token debit atomically. The issue is limited to inaccurate quoting and degraded integration behavior.

Recommendations

Derive `max_amount_1d` from the outbound limiter's currently available capacity after accounting for elapsed refill. Account for the relationship between the user-supplied amount and the post-fee, dust-adjusted `amount_received_1d` consumed by `send`.

Reuse the same fee, conversion, and rate-limit calculations in quoting and sending where possible to prevent the two paths from diverging. Continue returning the unrestricted maximum when no outbound limiter is configured.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd., and a fix was implemented in commit [2a7e7c44](#).

3.8. Insufficient test coverage

Target	Project-wide		
Category	Protocol Risks	Severity	Medium
Likelihood	N/A	Impact	Medium

Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test coverage for this project should be expanded to include all contracts, and all functions — not just surface-level functions. It is important to test the invariants required for ensuring security and also verify any mathematical properties from the project's specification. Additionally, testing cross-chain function calls and transfers is recommended to ensure the desired functionality.

Unit test cases that are missing include (but are not necessarily limited to) those documented in section [4.7](#).

Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits

are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

Recommendations

We recommend building a rigorous test suite that includes all contracts to ensure that the system operates securely and as intended.

Remediation

This issue has been acknowledged by Flare Ecosystems Ltd., and fixes were implemented in the following commits:

- [9373efa9](#) ↗
- [3fec9b83](#) ↗
- [b1371556](#) ↗

4. Threat model

This section documents the attack surface of security-critical EVM functions and Solana instructions. Each entry identifies externally controllable inputs, relevant state and account relationships, validation, and the ways an attacker could use those inputs to cause harm.

Not all in-scope functions or instructions have been modeled. The absence of a threat model does not indicate that a function or instruction is safe.

4.1. EVM

As time permitted, we analyzed the in-scope EVM contracts and created written threat models for critical functions. These entries focus on calldata, caller authorization, state transitions, inherited behavior, and security-relevant internal and external calls.

Module: FAssetOFTAdapter.sol

Function: initialize

Initializes a proxy instance by assigning the LayerZero delegate and contract owner to the supplied address and placing the pause module in its unpaused state. The initializer guard permits the proxy initialization only once.

Inputs

- `_owner`
 - **Validation:** Must be nonzero; the OApp core rejects a zero delegate before ownership is initialized. The function has no caller restriction, so safe deployment requires initialization to be performed atomically.
 - **Impact:** Becomes both the LayerZero endpoint delegate and the contract owner, gaining all owner-only OApp, fee, pause-sender, and rate-limit configuration powers.

Branches and code coverage

Intended branches

- ☐ A first proxy initialization with a nonzero `_owner` assigns that address as owner.
- ☐ A successful first initialization starts the proxy unpaused.
- ☐ A successful first initialization registers `_owner` as the OApp delegate at the immutable LayerZero endpoint.
- ☐ The fee parent initializer completes without introducing additional initial state.

- ☐ The pre-crime parent initializer completes without introducing additional initial state.
- ☐ The enforced-options parent initializer completes without introducing additional initial state.

Negative behavior

- ☐ A zero `_owner` reverts with `InvalidDelegate` before ownership or pause state is initialized.
- ☐ A second proxy initialization reverts with `InvalidInitialization` and preserves the original state.
- ☐ Calling `initialize` on the implementation reverts because its constructor disabled all initializers.
- ☐ If the endpoint rejects `setDelegate`, the whole initialization reverts without consuming version 1.
- ☐ Any account can initialize an uninitialized proxy and choose `_owner`, so a separately deployed uninitialized proxy can be taken over.

Function call analysis

- `initialize(address) -> __OFTAdapterFee_init(address)`
 - **External/internal?:** Internal initializer call.
 - **Argument control?:** `_owner` is fully caller-supplied.
 - **Impact:** Initializes the adapter/OApp hierarchy and invokes the fee initializer, which currently has no state to set.
- `__OFTAdapterFee_init(address) -> __OFTAdapter_init(address) -> __OFTCore_init(address) -> __OApp_init(address)`
 - **External/internal?:** Internal initializer chain.
 - **Argument control?:** The caller-supplied `_owner` is forwarded unchanged as the delegate.
 - **Impact:** Initializes the OApp sender and receiver hierarchy together with the pre-crime and enforced-options modules.
- `__OApp_init(address) -> __OAppCore_init(address) -> endpoint.setDelegate(address)`
 - **External/internal?:** Internal calls ending in an external call to the immutable LayerZero endpoint.
 - **Argument control?:** The endpoint address is fixed in the implementation constructor; the delegate is `_owner`.
 - **Impact:** Rejects a zero delegate and grants `_owner` endpoint-level OApp configuration authority; an endpoint revert rolls back initialization.
- `initialize(address) -> __Ownable_init(address) -> _transferOwnership(address)`
 - **External/internal?:** Internal initializer calls.

- **Argument control?:** `_owner` is fully caller-supplied after passing the delegate check.
- **Impact:** Stores `_owner` in the shared Ownable namespace and emits `OwnershipTransferred`.
- `initialize(address) -> __Pausable_init()`
 - **External/internal?:** Internal initializer call.
 - **Argument control?:** None.
 - **Impact:** Stores `paused = false`, enabling guarded debits and credits immediately after initialization.

Function: `quoteOFT`

Returns the adapter's current outgoing limit, the amounts that its fee-aware debit path would send and deliver, and a single fee line containing the full sent-versus-received difference. The advertised maximum is the lesser of current outgoing capacity and underlying-token total supply.

Inputs

- `_sendParam`
 - **Validation:** `amountLD` and `minAmountLD` are passed to the inherited debit preview, which reverts when the post-fee, de-dusted received amount is below `minAmountLD`. `dstEid` selects rate-limit and fee state. `to`, `extraOptions`, `composeMsg`, and `oftCmd` are not used by this quote.
 - **Impact:** Determines the quoted outgoing capacity, fee basis points, sent amount, received amount, and whether slippage makes the quote revert.

Branches and code coverage

Intended branches

- ☒ A configured destination whose remaining outgoing capacity is below total supply reports that capacity as `maxAmountLD`.
- ☒ If underlying-token total supply is below the remaining outgoing capacity, total supply is reported as `maxAmountLD`.
- ☒ Prior outgoing consumption reduces the remaining capacity reported by a later quote.
- ☐ A zero per-EID rate-limit window falls back to the default outgoing limit when that default has a nonzero window.
- ☒ If neither a per-EID nor default outgoing window is configured, unlimited rate capacity is reduced to the underlying-token total supply.
- ☐ An enabled destination-specific fee configuration takes precedence over the default fee basis points.

- ☒ A zero sent-versus-received difference returns an empty fee-detail array.
- ☒ A positive sent-versus-received difference returns one Flare system fee item equal to that complete difference.
- ☐ With a decimal conversion rate above one, de-dust remainder is included together with any BPS fee in the single fee item.

Negative behavior

- ☐ If the post-fee, de-dusted amount is below `minAmountLD`, the quote reverts with `SlippageExceeded`.
- ☐ An `amountLD` above the reported `maxAmountLD` is still quoted because the limit is informational and is enforced only during send.
- ☐ Pausing does not block this view function or change its quote.
- ☐ A revert from the immutable underlying token's `totalSupply` call propagates and prevents a quote.

Function call analysis

- `quoteOFT(SendParam) -> getAmountCanBeSent(uint32)`
 - **External/internal?:** Internal dispatch to a public view function.
 - **Argument control?:** `dstEid` is caller-supplied; capacity state and default configuration are owner-controlled.
 - **Impact:** Computes current outgoing consumption after linear decay and the destination's remaining capacity without recording a transfer.
- `getAmountCanBeSent(uint32) -> _effectiveOutgoing(uint192,uint64) -> _amountCanBeTransferred(uint192,uint64,uint192,uint64)`
 - **External/internal?:** Internal private/view calls; `_amountCanBeTransferred` is skipped when the effective window is zero.
 - **Argument control?:** Arguments derive from the caller-selected destination's stored state and owner-configured default.
 - **Impact:** Selects per-EID or default limits, applies time-based decay, and returns unlimited capacity when rate limiting is disabled.
- `quoteOFT(SendParam) -> innerToken.totalSupply()`
 - **External/internal?:** External view call to the immutable underlying ERC-20 token.
 - **Argument control?:** No arguments; the token address is fixed in the implementation constructor.
 - **Impact:** Supplies the second upper bound for `maxAmountLD`; a token revert propagates.
- `quoteOFT(SendParam) -> Math.min(uint256,uint256)`
 - **External/internal?:** Internal pure library call.

- **Argument control?:** One value is computed from owner-configured rate limits and time, and the other is returned by the underlying token.
 - **Impact:** Chooses the advertised outgoing maximum without enforcing it against amountLD.
- quoteOFT(SendParam) -> _debitView(uint256,uint256,uint32)
 - **External/internal?:** Internal inherited view call.
 - **Argument control?:** amountLD, minAmountLD, and dstEid are caller-supplied.
 - **Impact:** Keeps the requested amount as amountSentLD, subtracts the destination fee, removes unrepresentable dust from the received amount, and enforces slippage.
- _debitView(uint256,uint256,uint32) -> getFee(uint32,uint256) -> _getFeeBps(uint32)
 - **External/internal?:** Internal view calls.
 - **Argument control?:** The caller chooses the destination and amount; the owner controls default and destination-specific fee configuration.
 - **Impact:** Selects the enabled per-destination fee or default fee and calculates the BPS deduction.
- _debitView(uint256,uint256,uint32) -> _removeDust(uint256)
 - **External/internal?:** Internal view call.
 - **Argument control?:** The amount is the caller's requested amount after the configured fee; the conversion rate is immutable.
 - **Impact:** Rounds the deliverable amount down to shared-decimal precision. quoteOFT reports the resulting dust together with the BPS fee and does not move tokens or consume rate-limit capacity.

Module: FAssetOFT.sol

Function: initialize

Initializes a proxy with its ERC-20 metadata, owner, LayerZero delegate, fee/OApp state, and an unpaused state. The implementation contract disables initialization in its constructor, so this entry point is intended for delegate calls through a proxy.

Inputs

- _name
 - **Validation:** No content or length validation is performed; an empty string is accepted.

- **Impact:** Sets the inherited ERC-20 name.
- `_symbol`
 - **Validation:** No content or length validation is performed; an empty string is accepted.
 - **Impact:** Sets the inherited ERC-20 symbol.
- `_owner`
 - **Validation:** Must be nonzero; `__Ownable_init` rejects the zero address before the OApp delegate is configured.
 - **Impact:** Becomes both the contract owner and the delegate registered with the immutable LayerZero endpoint.

Branches and code coverage

Intended branches

- ☒ Proxy deployment with initializer calldata completes without reverting.
- ☐ Nonempty name and symbol values are stored verbatim.
- ☐ An empty name is accepted and stored.
- ☐ An empty symbol is accepted and stored.
- ☐ The nonzero owner is installed as the owner and registered as the endpoint delegate.
- ☐ Pausable state is explicitly initialized to unpaused.

Negative behavior

- ☐ A zero owner reverts with `OwnableInvalidOwner` before metadata or delegate setup is retained.
- ☐ A second top-level call after version one reverts with `InvalidInitialization` without changing state.
- ☐ Calling `initialize` directly on the implementation reverts because its constructor disabled initializers.
- ☐ A revert from `endpoint.setDelegate` reverts the complete initialization, including ownership and metadata writes.

Function call analysis

- `initialize(string,string,address) -> __Ownable_init(address)`
 - **External/internal?:** Internal inherited initializer call.
 - **Argument control?:** The caller fully controls `_owner`.
 - **Impact:** Rejects a zero owner, stores the initial owner, and emits `OwnershipTransferred`.
- `initialize(string,string,address) -> __OFTFee_init(string,string,address)`

- **External/internal?:** Internal inherited initializer call.
- **Argument control?:** All arguments come directly from the caller.
- **Impact:** Initializes ERC-20 metadata, OFT/OApp state, and fee state, and uses `_owner` as the LayerZero delegate.
- `__OFTFee_init(string,string,address) -> __ERC20_init(string,string)`
 - **External/internal?:** Internal inherited initializer call through `__OFT_init`.
 - **Argument control?:** The caller supplies both strings without contract-side normalization.
 - **Impact:** Writes the token name and symbol into ERC-7201 ERC-20 storage.
- `__OFTFee_init(string,string,address) -> endpoint.setDelegate(address)`
 - **External/internal?:** External call to the immutable LayerZero endpoint through the OFT/OApp initialization chain.
 - **Argument control?:** The caller supplies the delegate as `_owner`; the endpoint address was fixed on the implementation at construction.
 - **Impact:** Grants the selected owner authority over this OApp's endpoint configuration; a revert aborts initialization atomically.
- `initialize(string,string,address) -> __Pausable_init()`
 - **External/internal?:** Internal inherited initializer call.
 - **Argument control?:** No arguments.
 - **Impact:** Writes the initial unpaused state used by the customized debit and credit guards.

Function: quoteOFT

Reports the current send limit, fee detail, and debit preview for a native OFT transfer. The advertised maximum is the lower of the destination's remaining outgoing capacity and current token supply, while the receipt applies the configured fee and local-to-shared decimal de-dusting.

Inputs

- `_sendParam`
 - **Validation:** `amountLD` is used by the inherited fee and de-dusting preview, while `minAmountLD` is explicitly enforced against the resulting receive amount. `dstEid` selects rate-limit and fee state. `to`, `extraOptions`, `composeMsg`, and `oftCmd` are not read by this quote.
 - **Impact:** Selects the destination and proposed local-denomination amounts used to calculate the limit, fee details, and receipt.

Branches and code coverage

Intended branches

- ☐ If remaining outgoing capacity is below total supply, it determines `oftLimit.maxAmountLD`.
- ☒ If total supply is below remaining outgoing capacity, total supply determines `oftLimit.maxAmountLD`.
- ☐ If total supply equals remaining outgoing capacity, that shared value is returned as `oftLimit.maxAmountLD`.
- ☐ If the destination has no per-EID window, the default outgoing configuration determines its remaining capacity.
- ☐ If outgoing rate limiting is disabled for the destination, unlimited rate capacity leaves total supply as the binding maximum.
- ☐ An enabled destination-specific fee configuration takes precedence over the default fee.
- ☐ The default fee applies when no destination-specific fee is enabled.
- ☐ If fee and de-dusting remove nothing, sent and received amounts are equal and the fee-detail array is empty.
- ☐ If fee or de-dusting makes the sent amount larger than the received amount, their whole difference is reported as one Flare system fee item.
- ☐ The function remains callable while paused because it uses `_debitView`, not the pause-guarded `_debit`.

Negative behavior

- ☐ If the post-fee, de-dusted receive amount is below `minAmountLD`, the quote reverts with `SlippageExceeded`.
- ☐ Fee multiplication overflow reverts for an excessively large proposed amount.
- ☐ Missing destination peer configuration does not affect this quote, even though a later send to that destination reverts.
- ☐ A proposed amount above `oftLimit.maxAmountLD` is still previewed; this function reports the limit but does not enforce supply or rate capacity against `amountLD`.
- ☐ A receive preview above the shared-decimal `uint64` bound can still be returned because this function does not call `_toSD`; a later send reverts during encoding.

Function call analysis

- `quoteOFT(SendParam) -> getAmountCanBeSent(uint32)`
 - **External/internal?:** Internal call to a public view function.
 - **Argument control?:** The caller controls the destination endpoint ID.
 - **Impact:** Applies destination-specific or default outgoing configuration and linear decay to report remaining capacity without changing state.

- `quoteOFT(SendParam) -> totalSupply()`
 - **External/internal?:** Internal call to an inherited public view function.
 - **Argument control?:** No arguments; the value comes from current ERC-20 supply.
 - **Impact:** Prevents the advertised maximum from exceeding the tokens that currently exist on this chain.
- `quoteOFT(SendParam) -> Math.min(uint256,uint256)`
 - **External/internal?:** Internal library call.
 - **Argument control?:** Inputs are the calculated remaining capacity and current total supply.
 - **Impact:** Selects the binding advertised maximum.
- `quoteOFT(SendParam) -> _debitView(uint256,uint256,uint32)`
 - **External/internal?:** Internal inherited view call.
 - **Argument control?:** The caller supplies all three values through `_sendParam`; fee configuration is owner-controlled.
 - **Impact:** Calculates the full sent amount, fee-adjusted and de-dusted receive amount, and enforces the minimum receive amount without changing state.
- `_debitView(uint256,uint256,uint32) -> getFee(uint32,uint256)`
 - **External/internal?:** Internal call to an inherited public view function.
 - **Argument control?:** The caller controls destination and amount; the owner controls default and per-destination fee rates.
 - **Impact:** Selects the effective BPS fee and rounds its multiplication down to whole local units.
- `_debitView(uint256,uint256,uint32) -> _removeDust(uint256)`
 - **External/internal?:** Internal inherited view call.
 - **Argument control?:** The input is the proposed amount less the calculated fee.
 - **Impact:** Rounds the receive amount down to a multiple of the immutable local-to-shared decimal conversion rate.

Module: ExternalPausable.sol

Function: pause

Allows the owner or a registered pause sender to enter the paused state, blocking operations guarded by `whenNotPaused` until the contract is unpaused.

Inputs

This function has no explicit inputs.

Branches and code coverage

Intended branches

- ☒ The owner is implicitly authorized even when absent from the pause sender set.
- ☒ A non-owner registered in the pause sender set is authorized.
- ☒ A successful call while unpaused changes the paused state from false to true.
- ☒ Once paused, operations guarded by whenNotPaused revert with EnforcedPause.
- ☒ A successful call emits exactly one Paused event identifying the caller rather than the owner.

Negative behavior

- ☒ A non-owner absent from the pause sender set reverts with NotPauseSender.
- ☒ An authorized call while already paused reverts with EnforcedPause rather than succeeding idempotently.
- ☐ An unauthorized call while already paused reverts with NotPauseSender before the inherited paused-state check.

Function call analysis

- `pause()` -> `owner()`
 - **External/internal?:** Internal call to the inherited public view function.
 - **Argument control?:** No arguments are supplied; the result comes from inherited ownership storage.
 - **Impact:** Authorizes the owner without requiring membership in the explicit pause sender set.
- `pause()` -> `EnumerableSet.contains(address)`
 - **External/internal?:** Internal library view call, reached only when the caller is not the owner because the authorization expression short-circuits.
 - **Argument control?:** The checked address is `msg.sender`.
 - **Impact:** Determines whether a non-owner is registered as a pause sender; failure causes `NotPauseSender` before `_pause` executes.
- `pause()` -> `PausableUpgradeable._pause()`
 - **External/internal?:** Internal inherited call.
 - **Argument control?:** No arguments are supplied; the inherited implementation derives the event account from `_msgSender()`.

- **Impact:** Requires the contract to be unpaused, writes `true` to the inherited ERC-7201 paused-state slot, and emits `Paused` with the caller.

Function: `unpause`

Allows the owner or a registered pause sender to leave the paused state, re-enabling operations guarded by `whenNotPaused`.

Inputs

This function has no explicit inputs.

Branches and code coverage

Intended branches

- ☒ The owner is authorized to call `unpause`.
- ☒ A non-owner registered in the pause sender set is authorized.
- ☒ A successful call while paused changes the paused state from `true` to `false`.
- ☒ After unpausing, operations guarded by `whenNotPaused` can execute again.
- ☒ A successful owner call emits exactly one `Unpaused` event identifying the caller.

Negative behavior

- ☒ A non-owner absent from the pause sender set reverts with `NotPauseSender` while the contract is paused.
- ☒ An authorized call while not paused reverts with `ExpectedPause` rather than succeeding idempotently.
- ☐ An unauthorized call while not paused reverts with `NotPauseSender` before the inherited paused-state check.
- ☐ A registered non-owner's successful call emits `Unpaused` identifying that pause sender.

Function call analysis

- `unpause()` -> `owner()`
 - **External/internal?:** Internal call to the inherited public view function.
 - **Argument control?:** No arguments are supplied; the result comes from inherited ownership storage.
 - **Impact:** Authorises the owner without requiring membership in the explicit pause sender set.
- `unpause()` -> `EnumerableSet.contains(address)`

- **External/internal?:** Internal library view call, reached only when the caller is not the owner because the authorization expression short-circuits.
- **Argument control?:** The checked address is `msg.sender`.
- **Impact:** Determines whether a non-owner is registered as a pause sender; failure causes `NotPauseSender` before `_unpause` executes.
- `unpause()` -> `PausableUpgradeable._unpause()`
 - **External/internal?:** Internal inherited call.
 - **Argument control?:** No arguments are supplied; the inherited implementation derives the event account from `_msgSender()`.
 - **Impact:** Requires the contract to be paused, writes `false` to the inherited ERC-7201 paused-state slot, and emits `Unpaused` with the caller.

Function: `addPauseSender`

Allows the owner to add an address to the set of non-owner accounts authorized to call pause and unpause.

Inputs

- `_sender`
 - **Validation:** The caller must be the owner. The address itself is not validated, so the zero address, the owner, an existing member, or any other address is accepted.
 - **Impact:** Selects the address whose explicit pause and unpause authority is added.

Branches and code coverage

Intended branches

- ☒ A missing nonzero address is inserted into the pause sender set.
- ☒ A newly inserted address can subsequently call both pause and unpause.
- ☒ Adding an existing member again is idempotent and leaves a single set entry.
- ☐ The zero address can be inserted as a set member, although it cannot originate a normal transaction.
- ☐ Adding the owner inserts it into the enumerated set even though the owner is already implicitly authorized.
- ☐ A successful insertion emits no dedicated pause-sender membership event.

Negative behavior

- ☒ A non-owner call reverts with `OwnableUnauthorizedAccount`.

Function call analysis

- `addPauseSender(address) -> OwnableUpgradeable._checkOwner()`
 - **External/internal?:** Internal inherited call through the `onlyOwner` modifier.
 - **Argument control?:** No explicit arguments are supplied; the check compares inherited ownership storage with `_msgSender()`.
 - **Impact:** Reverts with `OwnableUnauthorizedAccount` unless the caller is the current owner, preventing unauthorized membership grants.
- `addPauseSender(address) -> EnumerableSet.add(address)`
 - **External/internal?:** Internal library call.
 - **Argument control?:** The owner fully controls `_sender`.
 - **Impact:** Converts the address to a `bytes32` key, checks whether it is already present, and appends it with a one-based position only when missing. The returned insertion flag is ignored, making duplicate additions no-ops.

Function: `removePauseSender`

Allows the owner to remove an address from the set of non-owner accounts authorized to call pause and unpaue.

Inputs

- `_sender`
 - **Validation:** The caller must be the owner. The address itself is not validated, and removing an address that is absent from the set is accepted.
 - **Impact:** Selects the address whose explicit pause and unpaue authority is revoked if present.

Branches and code coverage

Intended branches

- ☒ Removing an existing sole member pops it from the pause sender set.
- ☐ Removing an existing non-last member swaps the final member into its position before popping the final entry.
- ☒ A removed non-owner can no longer call pause.
- ☒ Removing an address that is not a member is idempotent and leaves the set unchanged.
- ☐ The zero address can be removed when it is present in the set.
- ☐ A successful removal emits no dedicated pause-sender membership event.

Negative behavior

- ☒ A non-owner call reverts with `OwnableUnauthorizedAccount`.
- ☐ Repeating a removal after the first successful removal returns normally as a missing-member no-op.

Function call analysis

- `removePauseSender(address) -> OwnableUpgradeable._checkOwner()`
 - **External/internal?:** Internal inherited call through the `onlyOwner` modifier.
 - **Argument control?:** No explicit arguments are supplied; the check compares inherited ownership storage with `_msgSender()`.
 - **Impact:** Reverts with `OwnableUnauthorizedAccount` unless the caller is the current owner, preventing unauthorized revocation.
- `removePauseSender(address) -> EnumerableSet.remove(address)`
 - **External/internal?:** Internal library call.
 - **Argument control?:** The owner fully controls `_sender`.
 - **Impact:** Looks up the address's one-based position. If present, it removes the entry in constant time by swapping in the final member when necessary, popping the array, and deleting the position; if absent, it returns false without changing storage. The return value is ignored.

Module: TransferLimiting.sol

Function: `getAmountCanBeSent`

Returns the destination's current consumed outgoing capacity after linear decay and the amount that can presently be sent. A destination-specific configuration takes precedence over the default outgoing configuration.

Inputs

- `_dstEid`
 - **Validation:** No explicit validation is required; every endpoint ID has either a destination-specific configuration, the default configuration, or unlimited capacity.
 - **Impact:** Selects the outgoing rate-limit state whose remaining capacity is reported.

Branches and code coverage

Intended branches

- ☒ A nonzero per-EID window uses that endpoint's configured limit and window.
- ☒ A zero per-EID window falls back to the default outgoing configuration.
- ☒ If the effective window is zero, rate limiting is disabled and the function returns `(0, type(uint256).max)`.
- ☒ If elapsed decay covers the consumed capacity, the current consumption is clamped to zero.
- ☒ Otherwise, elapsed decay is subtracted from the current consumed capacity.
- ☒ Remaining capacity is the effective limit minus current consumption, clamped to zero.

Negative behavior

- ☒ An unset EID follows the default or unlimited path without reverting.
- ☐ The zero-window path returns before `_amountCanBeTransferred`, preserving its nonzero-window precondition.

Function call analysis

- `getAmountCanBeSent(uint32) -> _effectiveOutgoing(uint192,uint64)`
 - **External/internal?:** Internal, private view call.
 - **Argument control?:** Arguments come from the selected endpoint's stored rate-limit state.
 - **Impact:** Chooses between the endpoint-specific and default outgoing configuration.
- `getAmountCanBeSent(uint32) -> _amountCanBeTransferred(uint192,uint64,uint192,uint64)`
 - **External/internal?:** Internal view call.
 - **Argument control?:** Arguments are the endpoint's stored consumption state and its effective owner-configured limit and window.
 - **Impact:** Applies linear decay and calculates the remaining sendable capacity without changing state.

Function: `getAmountCanBeReceived`

Returns the source's current consumed incoming capacity after linear decay and the amount that can presently be received. A source-specific configuration takes precedence over the default incoming configuration.

Inputs

- `_srcEid`
 - **Validation:** No explicit validation is required; every endpoint ID has either a source-specific configuration, the default configuration, or unlimited capacity.
 - **Impact:** Selects the incoming rate-limit state whose remaining capacity is reported.

Branches and code coverage

Intended branches

- ☒ A nonzero per-EID window uses that source's configured limit and window.
- ☐ A zero per-EID window falls back to the default incoming configuration.
- ☐ If the effective window is zero, rate limiting is disabled and the function returns `(0, type(uint256).max)`.
- ☐ If elapsed decay covers the consumed capacity, the current consumption is clamped to zero.
- ☐ Otherwise, elapsed decay is subtracted from the current consumed capacity.
- ☐ Remaining capacity is the effective limit minus current consumption, clamped to zero.

Negative behavior

- ☐ An unset EID follows the default or unlimited path without reverting.
- ☐ The zero-window path returns before `_amountCanBeTransferred`, preserving its nonzero-window precondition.

Function call analysis

- `getAmountCanBeReceived(uint32) -> _effectiveIncoming(uint192,uint64)`
 - **External/internal?:** Internal, private view call.
 - **Argument control?:** Arguments come from the selected endpoint's stored rate-limit state.
 - **Impact:** Chooses between the endpoint-specific and default incoming configuration.
- `getAmountCanBeReceived(uint32) -> _amountCanBeTransferred(uint192,uint64,uint192,uint64)`
 - **External/internal?:** Internal view call.
 - **Argument control?:** Arguments are the endpoint's stored consumption state and its effective owner-configured limit and window.

- **Impact:** Applies linear decay and calculates the remaining receivable capacity without changing state.

Function: `setOutgoingRateLimits`

Allows the owner to configure destination-specific outgoing rate limits. Existing consumption is first checkpointed against the previously effective configuration so that historical consumption is preserved across configuration changes.

Inputs

- `_configs`
 - **Validation:** The caller must be the owner. Individual entries accept any endpoint ID and limit. A nonzero window enables the destination-specific configuration, while a zero window intentionally returns the destination to the default outgoing configuration.
 - **Impact:** Controls the outgoing capacity and decay window applied to each listed destination.

Branches and code coverage

Intended branches

- ☐ An empty array leaves destination state unchanged and emits an empty change event.
- ☒ An existing nonzero per-EID window is checkpointed using the old per-EID configuration.
- ☒ An existing zero per-EID window is checkpointed using the old default configuration.
- ☒ A zero old effective window clears stale consumption.
- ☒ A nonzero old effective window decays consumption before the new configuration is stored.
- ☒ A new nonzero window installs a per-EID configuration; a zero window restores the default.
- ☒ A zero limit with a nonzero window blocks outgoing transfers to that destination.
- ☐ Duplicate EIDs are processed sequentially, and the final entry determines the stored configuration.

Negative behavior

- ☒ A non-owner call reverts before any configuration is changed.
- ☐ Failure while processing an entry reverts the entire batch and its event.
- ☐ An oversized array may run out of gas, reverting without partial updates.

Function call analysis

- `setOutgoingRateLimits(RateLimitConfig[]) -> _setOutgoingRateLimits(RateLimitConfig[])`
 - **External/internal?:** Internal call.
 - **Argument control?:** The owner fully controls the configuration array.
 - **Impact:** Iterates through the entries, checkpoints prior consumption, writes each destination's new configuration, and emits the change event.
- `_setOutgoingRateLimits(RateLimitConfig[]) -> _effectiveOutgoing(uint192,uint64)`
 - **External/internal?:** Internal, private view call.
 - **Argument control?:** Arguments come from the destination's old stored configuration.
 - **Impact:** Selects the old destination-specific or default configuration used for checkpointing.
- `_setOutgoingRateLimits(RateLimitConfig[]) -> _checkpoint(RateLimit,uint192,uint64)`
 - **External/internal?:** Internal, private call.
 - **Argument control?:** The storage entry is selected by the owner-supplied endpoint ID; limit and window are the old effective configuration.
 - **Impact:** Decays and preserves prior consumption before the new configuration takes effect.
- `_checkpoint(RateLimit,uint192,uint64) -> _amountCanBeTransferred(uint192,uint64,uint192,uint64)`
 - **External/internal?:** Internal view call, reached only for a nonzero old effective window.
 - **Argument control?:** Arguments derive from stored consumption and owner-configured rate-limit state.
 - **Impact:** Calculates the decayed consumption written by the checkpoint.

Function: `setIncomingRateLimits`

Allows the owner to configure source-specific incoming rate limits. Existing consumption is first checkpointed against the previously effective configuration so that historical consumption is preserved across configuration changes.

Inputs

- `_configs`

- **Validation:** The caller must be the owner. Individual entries accept any endpoint ID and limit. A nonzero window enables the source-specific configuration, while a zero window intentionally returns the source to the default incoming configuration.
- **Impact:** Controls the incoming capacity and decay window applied to each listed source.

Branches and code coverage

Intended branches

- ☐ An empty array leaves source state unchanged and emits an empty change event.
- ☐ An existing nonzero per-EID window is checkpointed using the old per-EID configuration.
- ☐ An existing zero per-EID window is checkpointed using the old default configuration.
- ☐ A zero old effective window clears stale consumption.
- ☐ A nonzero old effective window decays consumption before the new configuration is stored.
- ☐ A new nonzero window installs a per-EID configuration; a zero window restores the default.
- ☐ A zero limit with a nonzero window blocks incoming transfers from that source.
- ☐ Duplicate EIDs are processed sequentially, and the final entry determines the stored configuration.

Negative behavior

- ☐ A non-owner call reverts before any configuration is changed.
- ☐ Failure while processing an entry reverts the entire batch and its event.
- ☐ An oversized array may run out of gas, reverting without partial updates.

Function call analysis

- `setIncomingRateLimits(RateLimitConfig[]) -> _setIncomingRateLimits(RateLimitConfig[])`
 - **External/internal?:** Internal call.
 - **Argument control?:** The owner fully controls the configuration array.
 - **Impact:** Iterates through the entries, checkpoints prior consumption, writes each source's new configuration, and emits the change event.
- `_setIncomingRateLimits(RateLimitConfig[]) -> _effectiveIncoming(uint192,uint64)`
 - **External/internal?:** Internal, private view call.
 - **Argument control?:** Arguments come from the source's old stored

configuration.

- **Impact:** Selects the old source-specific or default configuration used for checkpointing.
- `_setIncomingRateLimits(RateLimitConfig[]) -> _checkpoint(RateLimit,uint192,uint64)`
 - **External/internal?:** Internal, private call.
 - **Argument control?:** The storage entry is selected by the owner-supplied endpoint ID; limit and window are the old effective configuration.
 - **Impact:** Decays and preserves prior consumption before the new configuration takes effect.
- `_checkpoint(RateLimit,uint192,uint64) -> _amountCanBeTransferred(uint192,uint64,uint192,uint64)`
 - **External/internal?:** Internal view call, reached only for a nonzero old effective window.
 - **Argument control?:** Arguments derive from stored consumption and owner-configured rate-limit state.
 - **Impact:** Calculates the decayed consumption written by the checkpoint.

Function: `setDefaultOutgoingRateLimit`

Allows the owner to replace the default outgoing rate limit used by every destination whose per-destination window is zero. Existing destination consumption is not checkpointed when the default changes.

Inputs

- `_limit`
 - **Validation:** The caller must be the owner. No numerical restriction is imposed; zero is a valid blocking limit when `_window` is nonzero.
 - **Impact:** Sets the maximum outgoing capacity for every destination using the default configuration.
- `_window`
 - **Validation:** The caller must be the owner. Zero is accepted and intentionally clears the default configuration.
 - **Impact:** Sets the default decay period. A zero value disables outgoing rate limiting for destinations without a destination-specific window.

Branches and code coverage

Intended branches

- ☒ A nonzero window installs the supplied default outgoing configuration.
- ☐ A zero window clears the default, making destinations without a per-EID window unlimited.
- ☐ A zero limit with a nonzero window blocks destinations using the default.
- ☒ A nonzero per-EID window continues to override the default.
- ☐ Changing the default does not checkpoint existing consumption; the next access uses the new default.
- ☒ `DefaultOutgoingRateLimitChanged` emits the new limit and window.

Negative behavior

- ☒ A non-owner call reverts before storage or events are changed.
- ☐ Zero-valued limits and windows are accepted with their documented semantics.

Function call analysis

No security-relevant function calls are made. The function writes directly to the ERC-7201 namespaced state and emits `DefaultOutgoingRateLimitChanged`.

Function: `setDefaultIncomingRateLimit`

Allows the owner to replace the default incoming rate limit used by every source whose per-source window is zero. Existing source consumption is not checkpointed when the default changes.

Inputs

- `_limit`
 - **Validation:** The caller must be the owner. No numerical restriction is imposed; zero is a valid blocking limit when `_window` is nonzero.
 - **Impact:** Sets the maximum incoming capacity for every source using the default configuration.
- `_window`
 - **Validation:** The caller must be the owner. Zero is accepted and intentionally clears the default configuration.
 - **Impact:** Sets the default decay period. A zero value disables incoming rate limiting for sources without a source-specific window.

Branches and code coverage

Intended branches

- ☒ A nonzero window installs the supplied default incoming configuration.
- ☐ A zero window clears the default, making sources without a per-EID window unlimited.
- ☐ A zero limit with a nonzero window blocks sources using the default.
- ☐ A nonzero per-EID window continues to override the default.
- ☐ Changing the default does not checkpoint existing consumption; the next access uses the new default.
- ☐ `DefaultIncomingRateLimitChanged` emits the new limit and window.

Negative behavior

- ☒ A non-owner call reverts before storage or events are changed.
- ☐ Zero-valued limits and windows are accepted with their documented semantics.

Function call analysis

No security-relevant function calls are made. The function writes directly to the ERC-7201 namespaced state and emits `DefaultIncomingRateLimitChanged`.

4.2. Solana

As time permitted, we analyzed the program's critical instructions. These entries describe each instruction's parameters, accounts, validation, state changes, and cross-program invocations.

For brevity, system accounts and well-known program accounts are omitted from the account lists. The instructions use Anchor account types that automatically validate the public keys of these accounts.

Discriminant, ownership, and rent checks are not discussed for each account unless they create a specific security concern. Unless otherwise stated, Anchor account types perform these checks automatically.

Program: `oft`

Instruction: `oft_version`

Returns the OFT interface and message-format versions.

Input parameters

None.

Accounts

None.

Additional checks and behavior

- Returns Version { interface: 2, message: 1 }.
- Does not read or modify program state.

CPI

None.

Test coverage

No test directly executes oft_version.

Instruction: init_oft

Creates an OFT instance for a token mint. It initializes the store, receive-types account, escrow, and Endpoint registration.

Input parameters

```
pub struct InitOFTParams {
  pub oft_type: OFTType,
  pub admin: Pubkey,
  pub shared_decimals: u8,
  pub endpoint_program: Option<Pubkey>,
}
```

- **oft_type**: Selects native or adapter mode.
- **admin**: Initial OFT admin and Endpoint delegate.
- **shared_decimals**: Cross-chain decimal precision.
- **endpoint_program**: Custom Endpoint, or the canonical Endpoint when None.

Accounts

- **payer**: Pays for all initialized accounts.

- Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **oft_store**: Stores OFT configuration and signs later CPIs.
 - Signer: No; the program signs with its PDA seeds.
 - Init: Yes.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: Yes.
- **lz_receive_types_accounts**: Stores the OFT store and mint used by account resolution.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, with seeds ["LzReceiveTypes", oft_store].
 - Mutable: Yes.
- **token_mint**: Mint used by the OFT.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **token_escrow**: Fresh token account that determines the OFT store address.
 - Signer: Yes.
 - Init: Yes.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Holds token_mint and uses oft_store as authority.
- **token_program**: SPL Token or Token-2022 program for the mint and escrow.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **oapp_registry**: Endpoint registry supplied through the remaining accounts.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, with seeds ["OApp", oft_store] under the selected Endpoint.
 - Mutable: Yes.

Additional checks and behavior

- `shared_decimals` cannot exceed the mint decimals.
- `ld2sd_rate` is $10^{(\text{mint_decimals} - \text{shared_decimals})}$.
- The selected OFT mode, mint, escrow, Endpoint, and conversion rate cannot be reconfigured later.
- The store starts with zero TVL and fees, unpaused, and without pause roles.
- The receive-types account stores `oft_store` and `token_mint`.
- The escrow starts empty and is created in both OFT modes.
- A distinct escrow key permits another OFT instance for the same mint.
- A zero admin is accepted and makes admin-only operations inaccessible.
- A custom Endpoint becomes the instance's messaging trust root.
- Any failure rolls back all initialized accounts and registration state.

CPI

- **Token account initialization:** Creates `token_escrow` for `token_mint` with `oft_store` as authority.
- **Endpoint register_oapp:** Registers `oft_store` and sets admin as delegate.

Test coverage

No test directly executes `init_oft`.

Recommended instruction-level coverage:

- Native and adapter initialization.
- SPL Token and Token-2022 mints.
- Decimal and custom-Endpoint branches.
- Initialized account state and rollback.

Instruction: `set_oft_config`

Updates OFT administration, fees, and pause configuration. Only the current admin may call it.

Input parameters

```
pub enum SetOFTConfigParams {
    Admin(Pubkey),
    Delegate(Pubkey),
```

```
DefaultFee(u16),
Paused(bool),
Pauser(Option<Pubkey>),
Unpauser(Option<Pubkey>),
}
```

- **Admin:** New OFT admin.
- **Delegate:** New Endpoint delegate for the OFT OApp.
- **DefaultFee:** Default outgoing OFT fee in basis points.
- **Paused:** New global pause state.
- **Pauser:** Optional key allowed to pause through `set_pause`.
- **Unpauser:** Optional key allowed to unpause through `set_pause`.

Accounts

- **admin:** Current OFT administrator. It authorizes every variant.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must equal `oft_store.admin`.
- **oft_store:** Stores the OFT configuration and acts as the Endpoint OApp.
 - Signer: No; the program signs the delegate CPI with its PDA seeds.
 - Init: No.
 - PDA: Yes, with seeds `["OFT", token_escrow]`.
 - Mutable: Yes.

Ordered remaining accounts support the Endpoint `set_delegate` CPI. They include the OApp registry and Endpoint event accounts.

Additional checks and behavior

- Every variant requires the current admin to sign.
- Admin replaces the admin immediately. The replacement does not accept the role, so an unusable address can lock admin operations.
- Delegate updates the Endpoint OApp delegate.
- The OFT admin and Endpoint delegate are independent.
- `DefaultFee` must be below 10,000 basis points.
- `Paused` directly sets the global pause state.
- `Pauser` and `Unpauser` set or clear the directional pause roles.

- This instruction remains available while paused.

CPI

- **Endpoint `set_delegate`:** Updates the OApp delegate. `oft_store` signs the CPI.

Test coverage

No test directly executes `set_oft_config`.

Recommended instruction-level coverage:

- Current-admin authorization and immediate admin handoff.
- All state variants, including fee boundaries and peer-fee precedence.
- Directional pause roles, role revocation, and direct admin pause changes.
- Successful delegate updates and invalid Endpoint remaining accounts.

Instruction: `set_peer_config`

Creates or updates one configuration field for a remote endpoint. Only the OFT admin may call it.

Input parameters

```
pub struct SetPeerConfigParams {
    pub remote_eid: u32,
    pub config: PeerConfigParam,
}

pub enum PeerConfigParam {
    PeerAddress([u8; 32]),
    FeeBps(Option<u16>),
    EnforcedOptions { send: Vec<u8>, send_and_call: Vec<u8> },
    OutboundRateLimit(Option<RateLimitParams>),
    InboundRateLimit(Option<RateLimitParams>),
}

pub struct RateLimitParams {
    pub refill_per_second: Option<u64>,
    pub capacity: Option<u64>,
}
```

- **`remote_eid`:** Remote endpoint ID and peer selector.

- **PeerAddress:** Replaces the remote OFT address.
- **FeeBps:** Sets or clears the route fee.
- **EnforcedOptions:** Replaces the plain-send and compose options.
- **OutboundRateLimit:** Updates or removes the outbound limiter.
- **InboundRateLimit:** Updates or removes the inbound limiter.
- **refill_per_second:** Optional refill-rate update.
- **capacity:** Optional capacity update.

Accounts

- **admin:** Authorizes the change and funds peer creation.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **peer:** Stores configuration for remote_eid.
 - Signer: No.
 - Init: If needed.
 - PDA: Yes, with seeds ["Peer", oft_store, remote_eid.to_be_bytes()].
 - Mutable: Yes.
- **oft_store:** Supplies the authoritative admin.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: No.
 - Constraints: Its admin must equal admin.

Additional checks and behavior

- Any variant may create the peer account.
- A new peer starts with a zero address, empty options, no fee override, and no limiters.
- A zero peer address is accepted. Sending through that route can burn or lock tokens without delivery.
- A route fee must be below 10,000 basis points. None restores the default fee.
- Both option blobs must use type 3. They replace the prior values together.
- The account allocation allows 512 bytes for send and 1,024 bytes for send_and_call.
- Inbound and outbound limiters are independent.
- An outer None removes the selected limiter.
- Omitted fields preserve an existing limiter's values. They remain zero on a new limiter.

- Updating capacity fills the bucket and resets its timestamp.
- Updating the refill rate first refills at the old rate.
- Capacity is updated before the refill rate when both are supplied.

CPI

None.

Test coverage

The generated size test covers only the enforced-options allocation. No test directly executes `set_peer_config`.

Recommended instruction-level coverage:

- Admin authorization and first-call initialization.
- Peer and fee boundary values.
- Option validation and allocation limits.
- Independent limiter updates and removal.

Instruction: `set_pause`

Sets the OFT pause state. Pausing and unpausing use separately configured signers.

Input parameters

```
pub struct SetPauseParams {
  pub paused: bool,
}
```

- **paused:** Requested pause state. `true` selects the pauser; `false` selects the unpauser.

Accounts

- **signer:** Authority selected by the requested pause state.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.

- Constraints: Must equal `oft_store.pause` when pausing or `oft_store.unpause` when unpausing.
- **oft_store**: Stores the configured pause authorities and current pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", `token_escrow`].
 - Mutable: Yes.

Additional checks and behavior

- Pausing requires the configured pauser.
- Unpausing requires the configured unpauser.
- An unset or mismatched authority fails with `Unauthorized`.
- A successful call stores the requested state. Repeating the current state is allowed.

CPI

None.

Test coverage

No test directly executes `set_pause`.

Recommended instruction-level coverage:

- Authorized pause and unpause with separate signers.
- Unset, mismatched, and cross-role authorities.
- Unchanged state after authorization failure.
- Repeated pause and unpause requests.

Instruction: `withdraw_fee`

Withdraws an admin-selected amount from the token escrow. The withdrawal is limited to the escrow balance not reserved by `tv1_1d`.

Input parameters

```
pub struct WithdrawFeeParams {
    pub fee_ld: u64,
}
```

- **fee_ld:** Local-decimal amount to transfer from the escrow. It must not exceed `token_escrow.amount - oft_store.tvl_ld`.

Accounts

- **admin:** Authorizes the fee withdrawal.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **oft_store:** Stores the admin, configured mint, escrow, TVL, and signing bump.
 - Signer: No; the program signs with its PDA seeds.
 - Init: No.
 - PDA: Yes, with seeds `["OFT", token_escrow]`.
 - Mutable: No.
 - Constraints: Its admin must equal admin.
- **token_mint:** Mint configured for this OFT instance.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must match `oft_store.token_mint` and be owned by `token_program`.
- **token_escrow:** Holds the configured mint's backing tokens and accrued fees.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must match `oft_store.token_escrow`, use `oft_store` as its authority, and hold `token_mint`.
- **token_dest:** Admin-selected token account that receives the withdrawal.
 - Signer: No.

- Init: No.
- PDA: No.
- Mutable: Yes.
- Constraints: Must hold `token_mint` and be owned by `token_program`.
- **token_program**: SPL Token or Token-2022 program for the mint and token accounts.
- Signer: No.
- Init: No.
- PDA: No.
- Mutable: No.

Additional checks and behavior

- `admin` must sign and match the `admin` stored in `oft_store`.
- The withdrawable balance is `token_escrow.amount - oft_store.tvl_ld`.
- A request above that balance fails with `InvalidFee`.
- The checked subtraction aborts if `tv1_ld` exceeds the escrow amount.
- The transfer uses `fee_ld` directly. It does not round the amount or update `tv1_ld`.
- The `admin` may select any destination token account for the configured mint.
- SPL Token and Token-2022 are supported through the token interface.
- Token-2022 transfer fees may make the destination's credited amount lower than `fee_ld`.

CPI

- **Token transfer_checked**: Transfers `fee_ld` from `token_escrow` to `token_dest` using the mint's decimals. `oft_store` signs with seeds `["OFT", token_escrow, bump]`.

Test coverage

No test directly executes `withdraw_fee`.

Recommended instruction-level coverage:

- Successful SPL Token and Token-2022 withdrawals, including transfer fees.
- The exact withdrawable-balance boundary and an `InvalidFee` over-withdrawal.
- Unauthorized admins and invalid mint, escrow, or destination relationships.
- Preservation of the balance reserved by `tv1_ld`.

Instruction: quote_of_t

Returns the token amounts and fees for a prospective send. It does not quote Endpoint messaging fees.

Input parameters

```
pub struct QuoteOFTParams {
    pub dst_eid: u32,
    pub to: [u8; 32],
    pub amount_ld: u64,
    pub min_amount_ld: u64,
    pub options: Vec<u8>,
    pub compose_msg: Option<Vec<u8>>,
    pub pay_in_lz_token: bool,
}
```

- **dst_eid**: Destination endpoint ID and peer selector.
- **to**: Remote recipient. Unused by this quote.
- **amount_ld**: Requested amount in local decimals.
- **min_amount_ld**: Minimum acceptable received amount.
- **options**: Unused by this quote.
- **compose_msg**: Unused by this quote.
- **pay_in_lz_token**: Unused by this quote.

Accounts

- **oft_store**: Supplies the OFT type, conversion rate, default fee, and pause state.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: No.
- **peer**: Supplies the destination fee override and outbound limiter.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["Peer", oft_store, dst_eid.to_be_bytes()].
 - Mutable: No.
- **token_mint**: Supplies the current Token-2022 transfer-fee configuration.
 - Signer: No.

- Init: No.
- PDA: No.
- Mutable: No.
- Constraints: Must match `oft_store.token_mint`.

Additional checks and behavior

- Rejects quotes while the OFT is paused.
- All amounts use local decimals.
- Adapter mode previews the transfer fee, removes shared-decimal dust, and derives the gross source debit.
- Native mode removes dust without applying a token transfer fee.
- A peer fee overrides the default OFT fee. The fee is rounded down and dusted.
- The received amount must satisfy `min_amount_ld`.
- The receipt returns the source debit and remotely deliverable amount.
- Positive token and OFT fees produce separate fee details.
- `min_amount_ld` in the returned limit is always zero. `max_amount_ld` is always `u64::MAX`.
- The reported maximum ignores the outbound rate limiter. A later send may therefore fail with `RateLimitExceeded`.

CPI

None.

Test coverage

No test directly executes `quote_oft`.

Recommended instruction-level coverage:

- Native and adapter quotes with dust and fees.
- Token-2022 transfer fees and arithmetic boundaries.
- Slippage, pause, and outbound-limit mismatches.

Instruction: `quote_send`

Returns the LayerZero messaging fee for a prospective send. It also previews the remotely deliverable token amount and enforces the slippage floor.

Input parameters

```
pub struct QuoteSendParams {
    pub dst_eid: u32,
    pub to: [u8; 32],
    pub amount_ld: u64,
    pub min_amount_ld: u64,
    pub options: Vec<u8>,
    pub compose_msg: Option<Vec<u8>>,
    pub pay_in_lz_token: bool,
}
```

- **dst_eid**: Destination endpoint ID and peer selector.
- **to**: Remote recipient encoded in the message.
- **amount_ld**: Requested amount in local decimals.
- **min_amount_ld**: Minimum acceptable received amount.
- **options**: Caller-supplied LayerZero options.
- **compose_msg**: Optional compose payload.
- **pay_in_lz_token**: Selects the Endpoint's LZ-token quote path.

Accounts

- **oft_store**: Supplies the OFT configuration and Endpoint program.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: No.
- **peer**: Supplies the remote peer, fee override, and enforced options.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["Peer", oft_store, dst_eid.to_be_bytes()].
 - Mutable: No.
- **token_mint**: Supplies decimals and Token-2022 transfer-fee data.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must match oft_store.token_mint.

Ordered remaining accounts supply the configured Endpoint, send-library configuration,

outbound nonce, and library quote accounts. The quote has no payer or user signer.

Additional checks and behavior

- Rejects quotes while the OFT is paused.
- Uses the same fee and dust preview as `quote_of`.
- The received amount must satisfy `min_amount_ld`.
- Plain and composed messages use the peer's `send` and `send_and_call` options, respectively.
- Caller options are combined with enforced type-3 options.
- The message amount should use shared decimals. This instruction instead encodes `amount_received_ld` directly, so quotes differ from `send` when `ld2sd_rate != 1`.
- The Endpoint selects the `send` library and returns its fee quote without changing state.

CPI

- **Endpoint quote:** Quotes the constructed packet and options through the selected `send` library. It returns `native_fee` and `lz_token_fee` unchanged.

Test coverage

Codec tests cover plain and composed message layouts. No test directly executes `quote_send`.

Recommended instruction-level coverage:

- Native and adapter amount previews.
- Slippage and option combinations.
- Shared-decimal parity with `send`.
- Native and LZ-token fee quotes.

Instruction: `send`

Debits the caller and sends an OFT packet to the configured peer. Adapter OFTs escrow tokens. Native OFTs burn the bridged amount and escrow the OFT fee.

Input parameters

```
pub struct SendParams {  
    pub dst_eid: u32,  
    pub to: [u8; 32],
```

```
pub amount_id: u64,
pub min_amount_id: u64,
pub options: Vec<u8>,
pub compose_msg: Option<Vec<u8>>,
pub native_fee: u64,
pub lz_token_fee: u64,
}
```

- **dst_id**: Destination endpoint ID and peer selector.
- **to**: Remote recipient encoded in the message.
- **amount_id**: Requested amount in local decimals.
- **min_amount_id**: Minimum remotely deliverable amount.
- **options**: Caller-supplied LayerZero options.
- **compose_msg**: Optional compose payload.
- **native_fee**: Native messaging-fee allowance.
- **lz_token_fee**: LZ-token fee allowance.

Accounts

- **signer**: Authorizes the source-token debit.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **peer**: Stores the destination configuration and outbound limiter.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["Peer", oft_store, dst_id.to_be_bytes()].
 - Mutable: Yes.
- **oft_store**: Stores OFT configuration, pause state, and adapter TVL.
 - Signer: No; the program signs with its PDA seeds.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: Yes.
- **token_source**: Token account debited by the send.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.

- Constraints: Must use signer as authority and hold `token_mint`.
- **token_escrow**: Receives adapter backing tokens or native-OFT fees.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must match `oft_store.token_escrow`, use `oft_store` as authority, and hold `token_mint`.
- **token_mint**: Mint configured for this OFT instance.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must match `oft_store.token_mint` and be owned by `token_program`.
- **token_program**: SPL Token or Token-2022 program for the mint and token accounts.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **event_authority**: Authority used to emit OFTSent.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seed "`__event_authority`".
 - Mutable: No.

Ordered remaining accounts support the Endpoint, selected send library, fee payment, and events. `oft_store` must occupy the Endpoint sender position.

Additional checks and behavior

- Rejects sends while the OFT is paused.
- A peer fee overrides the default OFT fee. Amounts are rounded to shared-decimal precision.
- The received amount must satisfy `min_amount_ld`.
- The outbound limiter consumes the remotely deliverable amount.
- Adapter mode transfers the gross source amount into escrow and adds the net bridged amount to `tv1_ld`.
- Native mode burns the bridged amount. A nonzero OFT fee is transferred into escrow.
- The message encodes the recipient and shared-decimal amount. Composed

messages also encode `signer` and the compose payload.

- Caller options are combined with the peer's enforced options.
- The Endpoint sends to `peer.peer_address` and returns its messaging receipt.
- An all-zero recipient is accepted and may strand tokens on the destination.
- Any failure rolls back rate limiting, token movement, TVL changes, fee payment, and Endpoint state.

CPI

- **Token transfer_checked**: Locks adapter tokens or transfers a native-OFT fee.
- **Token burn**: Burns the bridged amount in native mode.
- **Endpoint send**: Dispatches the packet through the configured send library and pays messaging fees.
- **Event CPI**: Emits `OFTSent`.

Test coverage

Codec tests cover plain and composed messages. No test directly executes `send`.

Recommended instruction-level coverage:

- Adapter and native sends with SPL Token and Token-2022.
- Dust, fees, slippage, and rate limiting.
- Plain and composed options.
- Native and LZ-token fee payment.
- Rollback after failed CPIs.

Instruction: `lz_receive`

Authenticates a LayerZero packet and credits its encoded recipient. Adapter OFTs release escrowed tokens; native OFTs mint tokens. The instruction also consumes the inbound rate limit and registers compose data with the Endpoint.

Input parameters

```
pub struct LzReceiveParams {
    pub src_eid: u32,
    pub sender: [u8; 32],
    pub nonce: u64,
    pub guid: [u8; 32],
    pub message: Vec<u8>,
```

```
pub extra_data: Vec<u8>,
}
```

- **src_eid**: Source endpoint ID and peer selector.
- **sender**: Remote OApp address. It must match the selected peer.
- **nonce**: Packet nonce validated by the Endpoint.
- **guid**: Packet identifier bound to message by the Endpoint.
- **message**: Encodes the recipient, shared-decimal amount, and optional compose data.
- **extra_data**: Unused.

Accounts

- **payer**: Pays to create the destination and compose-state accounts.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
- **peer**: Stores the authenticated remote peer and its inbound rate limiter.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["Peer", oft_store, src_eid.to_be_bytes()]
 - Mutable: Yes.
 - Constraints: Its peer_address must equal sender.
- **oft_store**: Stores the OFT configuration, pause state, and adapter TVL.
 - Signer: No; the program signs with its PDA seeds.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: Yes.
- **token_escrow**: Holds the tokens backing an adapter OFT.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must match oft_store.token_escrow, use oft_store as its authority, and hold token_mint.
- **to_address**: Recipient decoded from message.
 - Signer: No.
 - Init: No.

- PDA: No.
 - Mutable: No.
 - Constraints: Unchecked account whose address must match the decoded recipient.
- **token_dest**: Token account credited by the receive. payer funds its creation.
 - Signer: No.
 - Init: If needed.
 - PDA: Yes, the associated token account for `to_address` and `token_mint`.
 - Mutable: Yes.
- **token_mint**: Mint associated with this OFT instance.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must match `oft_store.token_mint` and be owned by `token_program`.
- **mint_authority**: Optional authority used in native mode.
 - Signer: No; `oft_store` signs the CPI.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must match `token_mint.mint_authority` when supplied.
- **token_program**: SPL Token or Token-2022 program for the mint and token accounts.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
- **event_authority**: Authority used to emit events through CPI.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seed "`__event_authority`".
 - Mutable: No.

Ordered remaining accounts support the Endpoint `clear` CPI and optional `send_compose` CPI. The Endpoint validates its registry, nonce, payload hash, settings, event authority, and compose state.

Additional checks and behavior

- Rejects packets while `oft_store.paused` is set.

- The selected peer authenticates sender for `src_eid`.
- Endpoint `clear` validates the receiver, nonce, and `keccak256(guid || message)`.
- Messages shorter than 40 bytes abort during decoding.
- Bytes after offset 40 are compose data. Their size is not limited.
- The shared-decimal amount is converted using `1d2sd_rate`. Overflow aborts.
- The inbound limiter consumes the pre-transfer-fee local amount.
- Adapter mode reduces `tv1_ld` and releases tokens from escrow. Token-2022 fees may reduce the credited amount.
- Native mode mints tokens using the configured authority and the `oft_store` signature.
- Compose data is registered at index zero with the actual credited amount.
- An all-zero recipient is accepted and may strand the tokens.
- A successful receive emits `OFTReceived`.
- Any failure rolls back payload clearing, accounting, token movement, and compose registration.

CPI

- **Endpoint `clear`:** Validates and consumes the verified payload. `oft_store` signs as the receiver.
- **Token `transfer_checked`:** Adapter mode releases tokens from escrow. Transfer fees may reduce the credited amount.
- **Token `mint_to`:** Native mode mints tokens to the destination account.
- **Endpoint `send_compose`:** Registers the composed-message state. payer funds its creation.
- **Event `CPI`:** Emits `OFTReceived`.

Test coverage

Codec tests cover messages with and without compose data. No test directly executes `lz_receive`.

Recommended instruction-level coverage:

- Successful adapter and native receives.
- Destination-account creation and Token-2022 transfer fees.
- Invalid peers, payloads, mint authorities, and exhausted rate limits.
- Paused, malformed, and overflowing receives.
- Compose registration and rollback after a failed CPI.

Instruction: lz_receive_types

Returns the ordered account metadata required by `lz_receive`. It derives token, Endpoint, and optional compose accounts without changing state.

Input parameters

```
pub struct LzReceiveParams {
    pub src_eid: u32,
    pub sender: [u8; 32],
    pub nonce: u64,
    pub guid: [u8; 32],
    pub message: Vec<u8>,
    pub extra_data: Vec<u8>,
}
```

- **src_eid**: Source endpoint ID and peer selector.
- **sender**: Remote OApp used in Endpoint PDA derivations.
- **nonce**: Packet nonce used in Endpoint PDA derivations.
- **guid**: Packet identifier used for compose state.
- **message**: Encodes the recipient, amount, and optional compose data.
- **extra_data**: Unused.

Accounts

- **oft_store**: Supplies the OFT, token, and Endpoint configuration.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with seeds ["OFT", token_escrow].
 - Mutable: No.
- **token_mint**: Supplies the token program, mint authority, and transfer-fee data.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must match `oft_store.token_mint`.

The returned accounts are ordered as follows:

- **Indices 0–3**: Executor payer, source peer, `oft_store`, and token escrow. The payer is a writable signer. The other three accounts are writable.

- **Indices 4–6:** Decoded recipient, recipient associated token account, and token mint. The token account and mint are writable.
- **Indices 7–12:** Mint authority, token program, Associated Token Program, System Program, OFT event authority, and OFT program. These accounts are read-only.
- **Indices 13–20:** Endpoint program, `oft_store` receiver, OApp registry, inbound nonce, payload hash, Endpoint settings, Endpoint event authority, and Endpoint event program. The nonce, payload hash, and settings are writable.
- **Indices 21–27:** Optional compose accounts. They are the Endpoint program, `oft_store` sender, executor payer, composed-message PDA, System Program, Endpoint event authority, and Endpoint event program. The payer is a writable signer and the composed-message PDA is writable.

Additional checks and behavior

- Decodes the recipient from bytes 0–31 and the shared-decimal amount from bytes 32–39.
- Derives the recipient associated token account using the mint's owner program.
- Derives Endpoint clear accounts from `src_eid`, sender, and nonce.
- Any bytes after offset 40 append the compose account group.
- Native compose data uses the converted local amount.
- Adapter compose data uses the post-transfer-fee amount.
- Messages shorter than 40 bytes abort during decoding.
- Returns 21 accounts without compose data and 28 with compose data.

CPI

None. Endpoint helpers only derive account metadata.

Test coverage

Codec tests cover message decoding and compose encoding. No test directly executes `lz_receive_types`.

Recommended instruction-level coverage:

- Exact account order and signer/writable flags.
- Native and adapter compose amounts.
- PDA derivation with and without compose data.

5. Assessment Results

During our assessment on the scoped Flare Solana OFT files, we discovered eight findings. No critical issues were found. Two findings were of medium impact, one was of low impact, and the remaining findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.